ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»

Параллельные вычисления

ЛАБОРАТОРНАЯ РАБОТА № 4

Тема: Измерение латентности кэш-памяти процессора и оперативной памяти

Группа: БПМ-22-ПО-1

Студент: Нургалиев Родион Дамирович

Ссылка на репозиторий с кодом:

https://github.com/LLrodyaLL/parallel_computing_misis_nurgaliev_r_d_bpm_22_po_1/tree/main/lab04

2026

1. Цель работы

Цель лабораторной работы - измерить латентность обращения к кэш-памяти различного уровня (L1d, L2, L3) и к оперативной памяти (RAM) для конкретной вычислительной системы в задаче нахождения суммы элементов вектора длины N. Реализовать три порядка обхода элементов массива: последовательный, случайный (генерация на лету) и случайный с предварительно вычисленным массивом индексов. Сравнить полученные результаты с эталонными значениями, приведёнными в методических указаниях.

2. Теоретические сведения

2.1. Иерархия памяти

Современные процессоры оснащены многоуровневой подсистемой памяти - "пирамидой памяти". Скорость доступа убывает, а ёмкость возрастает по мере удаления от ядра:

- Регистры процессора (<1 нс)
- Кэш L1d - наименьший по объёму (~32–64 КБ), работает на частоте ядра (~1–5 нс)
- Кэш L2 - промежуточный уровень (~256 КБ – 1 МБ, ~5–15 нс)
- Кэш L3 - разделяемый между ядрами (~8–64 МБ, ~15–50 нс)
- Оперативная память DRAM (~50–100+ нс)
- Внешняя память (SSD/HDD)

Обмен данными между уровнями происходит порциями размером 64 байта (кэш-линия). При последовательном доступе аппаратный механизм предвыборки (hardware prefetch) загружает следующую кэш-линию заблаговременно, скрывая задержку обращения к памяти. При случайном доступе предвыборка неэффективна: каждое обращение требует полной загрузки кэш-линии, а время итерации напрямую отражает латентность соответствующего уровня памяти.

2.2. Методика измерения

Для оценки латентности используется задача вычисления суммы элементов массива с накоплением:

```
S = 0;
for i from 1 to N do
    S = S + a[i];   // последовательный доступ
```

При случайном доступе механизм аппаратной предвыборки отключается, что позволяет измерить реальную латентность каждого уровня иерархии памяти. Измеряется время выполнения одной итерации цикла (нс/итерацию).

3. Описание реализации

3.1. Структура проекта

Программа реализована на языке C++ (стандарт C++17), собирается при помощи CMake 3.16+ и компилятора MSVC (Visual Studio 2022) с флагом оптимизации /O2. Визуализация результатов выполнена на Python с использованием библиотеки Matplotlib.

Структура проекта:

```
lab04-memory-latency/
├── include/
│   ├── benchmark.h
│   ├── csv_writer.h
│   └── utils.h
├── src/
│   ├── main.cpp
│   ├── benchmark.cpp
│   ├── csv_writer.cpp
│   └── utils.cpp
├── scripts/
│   └── plot.py
├── results/
├── CMakeLists.txt
└── .gitignore
```

3.2. Режимы доступа к памяти

Реализованы три режима доступа к элементам массива:

1. Последовательный (sequential) - обращение к элементам в порядке возрастания индексов. Аппаратная предвыборка работает эффективно; измеренная задержка отражает пропускную способность подсистемы памяти, а не латентность.
2. Случайный на лету (random_on_the_fly) - индекс генерируется функцией mt19937 на каждой итерации. Аппаратная предвыборка не работает. Полученное время включает как задержку доступа к памяти, так и накладные расходы генератора псевдослучайных чисел.
3. Случайный с предвычисленными индексами (random_precomputed) - массив случайных индексов формируется заранее (shuffle методом Фишера-Йейтса). Рабочий цикл использует только готовые индексы. Накладные расходы ГПСЧ исключены, однако хранение массива индексов удваивает объём используемой памяти.

3.3. Ключевые фрагменты кода

Ядро функции последовательного суммирования (benchmark.cpp):

```
std::uint64_t sequential_sum(const std::vector<std::int32_t>& data,
                             std::size_t repeats) {
    std::uint64_t sum = 0;
    for (std::size_t r = 0; r < repeats; ++r)
        for (std::size_t i = 0; i < data.size(); ++i)
            sum += static_cast<std::uint64_t>(data[i]);
    return sum;
}
```

Ядро функции случайного суммирования с предвычисленными индексами:

```
std::uint64_t random_sum_precomputed(
    const std::vector<std::int32_t>& data,
    const std::vector<std::size_t>& indices,
    std::size_t repeats) {
    std::uint64_t sum = 0;
    for (std::size_t r = 0; r < repeats; ++r)
        for (std::size_t i = 0; i < indices.size(); ++i)
            sum += static_cast<std::uint64_t>(data[indices[i]]);
    return sum;
}
```

Измерение времени выполнено через std::chrono::high_resolution_clock. Для предотвращения оптимизации компилятором результат суммирования записывается в volatile-переменную (do_not_optimize). Количество повторений (repeats) выбирается автоматически в зависимости от объёма данных: 200 для массивов до 2 МБ, 50 до 32 МБ и 10 для больших массивов.

3.4. Диапазоны тестирования

Диапазоны объёмов тестируемых данных:

Диапазон	Шаг	Цель измерения
1 КБ — 2 048 КБ	1 КБ	Кэши L1d и L2
2 560 КБ — 32 768 КБ	512 КБ	Кэши L2 и L3
37 МБ — 150 МБ	5 МБ	Кэш L3 и RAM

4. Результаты эксперимента

4.1. Последовательный доступ

При последовательном обходе массива аппаратный механизм предвыборки (hardware prefetch) эффективно скрывает латентность доступа к памяти, заблаговременно подгружая следующую кэш-линию. Время одной итерации остаётся практически постоянным (~0,15–0,30 нс) вне зависимости от объёма массива, поскольку производительность определяется пропускной способностью подсистемы памяти, а не её латентностью. Единственный всплеск (~1,26 нс) при малых объёмах объясняется накладными расходами запуска теста и прогрева кэша.

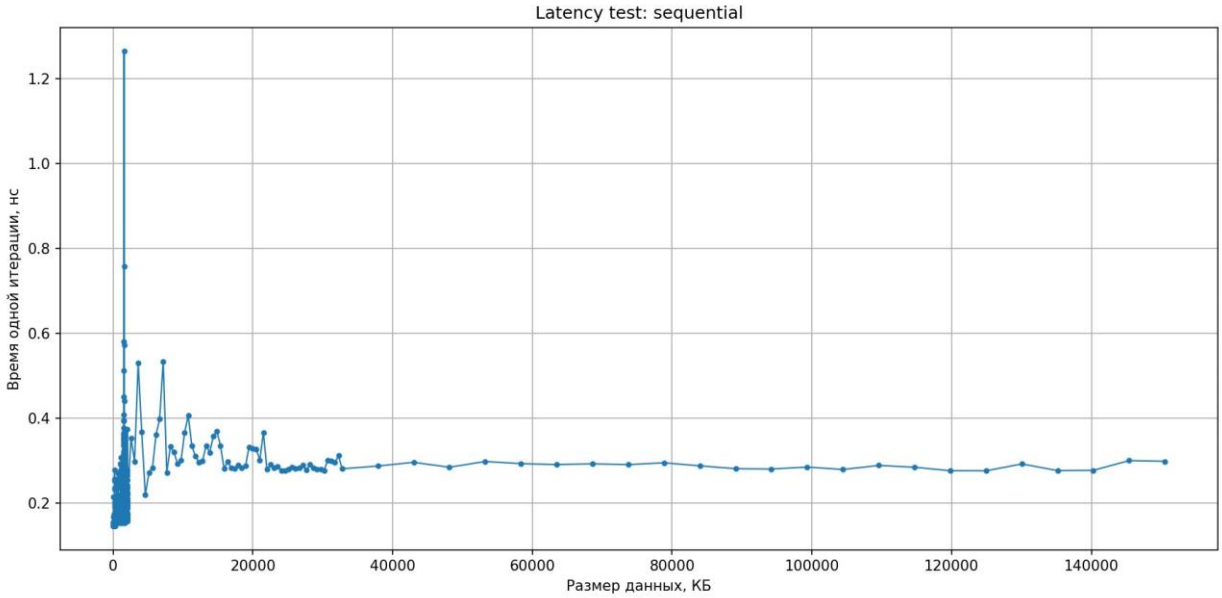

Рис. 1. Последовательный доступ: время одной итерации от объёма данных

4.2. Случайный доступ (генерация индексов на лету)

При случайном доступе аппаратная предвыборка не работает. Рост задержки отчётливо отражает переходы между уровнями иерархии памяти. При малых объёмах (≤64 КБ) данные целиком помещаются в кэш L1d, время итерации составляет ~7–8 нс. При

превышении объёма L1d (≈64 КБ) задержка возрастает до ~8–10 нс (кэш L2). Дальнейший рост соответствует вытеснению данных в кэш L3 (~10–20 нс) и в RAM (~32–34 нс при объёме свыше 32 МБ). Необходимо учитывать, что измеренные значения включают накладные расходы генератора псевдослучайных чисел (mt19937), что несколько завышает результат.

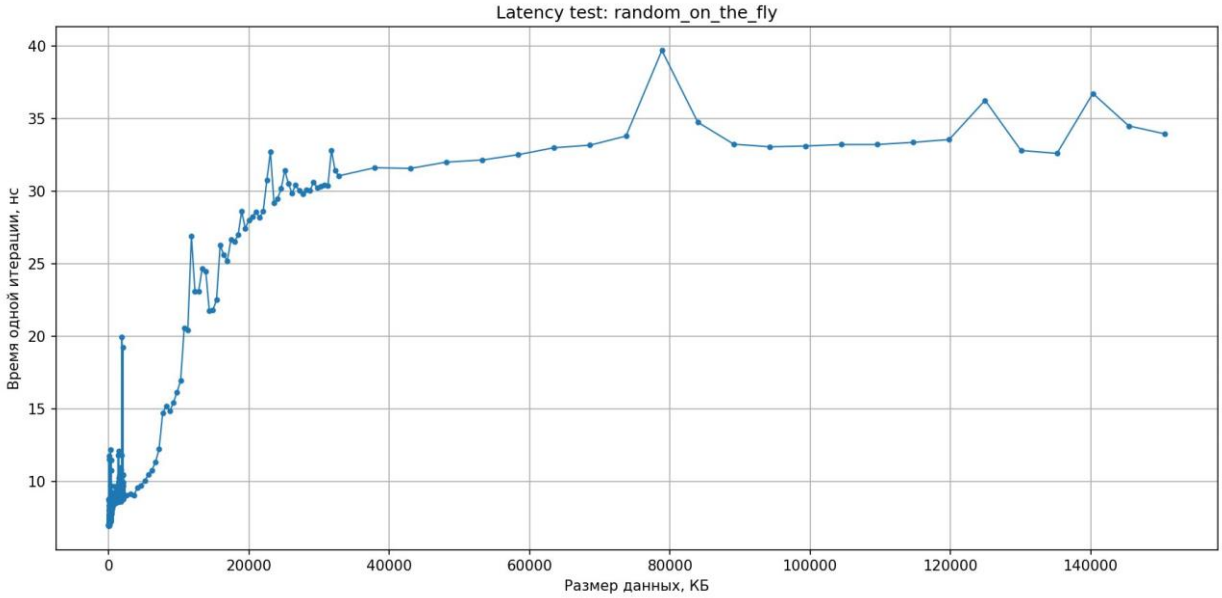

Рис. 2. Случайный доступ (генерация на лету): время одной итерации от объёма данных

4.3. Случайный доступ с предвычисленными индексами

Исключение накладных расходов ГПСЧ позволяет получить более точную оценку латентности. При объёме массива до ~64 КБ время итерации составляет ~0,3–0,4 нс (L1d). При выходе за пределы L1d задержка возрастает до ~1,0–1,2 нс (L2), далее плавно увеличивается до ~5–7 нс (L3), а при объёме свыше 32 МБ стабилизируется на уровне ~9–10 нс (RAM). Следует учитывать, что дополнительный массив индексов удваивает фактически используемый объём памяти, что несколько сдвигает точки перехода между уровнями.

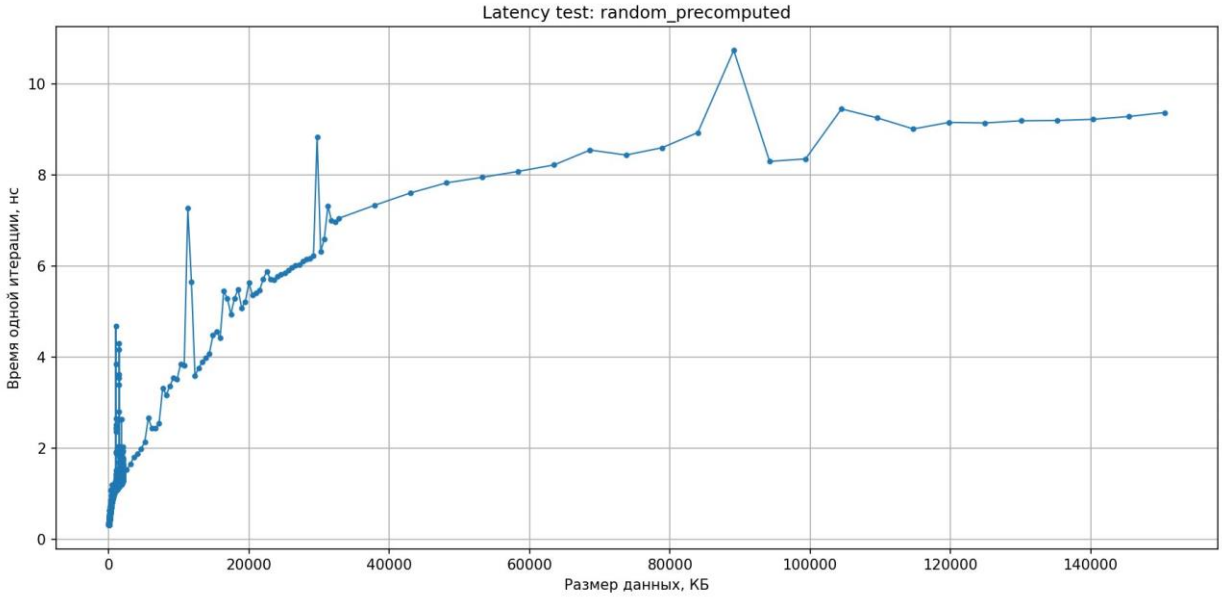

Рис. 3. Случайный доступ с предвычисленными индексами: время одной итерации от объёма данных

5. Сводные таблицы латентностей

5.1. Последовательный порядок обращения

Уровень памяти	Время итерации, нс
Весь диапазон (1 КБ – 150 МБ)	~0,15 – 0,30

Вывод: при последовательном доступе время итерации практически не зависит от объёма данных - hardware prefetch полностью скрывает латентность доступа к памяти.

5.2. Случайный порядок (генерация индексов на лету)

Кэш L1d (≤64 КБ)	Кэш L2 (64–1024 КБ)	Кэш L3 (1–32 МБ)	RAM (>32 МБ)
~7 – 8 нс	~8 – 12 нс	~10 – 33 нс	~32 – 34 нс

5.3. Случайный порядок с предвычисленными индексами

Кэш L1d (≤64 КБ)	Кэш L2 (64–1024 КБ)	Кэш L3 (1–32 МБ)	RAM (>32 МБ)
~0,3 – 0,4 нс	~0,4 – 1,2 нс	~1,2 – 7,0 нс	~9 – 10 нс

5.4. Сравнение с эталонными значениями

В таблице ниже приведено сравнение полученных результатов с эталонными данными для Intel Core i7 4770 из методических указаний. Измерения для данной работы выполнены в наносекундах и не привязаны к конкретной тактовой частоте, поэтому для сопоставления используется качественное сравнение соотношений задержек.

Режим / уровень	L1d	L2	L3	RAM
Эталон (i7 4770), такт	9	9	14–16	до 106
Получено, нс (precomputed)	~0,3–0,4	~0,4–1,2	~1,2–7,0	~9–10

6. Выводы

1. Разработана программа, измеряющая латентность доступа к кэш-памяти и RAM тремя методами. Реализованы последовательный, случайный и случайный с предвычисленными индексами порядки обхода; результаты сохраняются в CSV-файл и визуализируются скриптом на Python.
2. При последовательном обходе аппаратный механизм предвыборки полностью скрывает латентность подсистемы памяти: время итерации составляет ~0,15–0,30 нс и не зависит от объёма массива.
3. Случайный доступ выявляет реальную латентность каждого уровня иерархии. Режим с предвычисленными индексами исключает накладные расходы генератора ПСЧ и даёт более точные оценки: L1d — ~0,3–0,4 нс, L2 — ~0,4–1,2 нс, L3 — ~1,2–7,0 нс, RAM — ~9–10 нс.

4. Качественный характер кривых соответствует эталонным данным методических указаний: при последовательном доступе задержка практически не зависит от объёма, а при случайном — наблюдаются отчётливые ступенчатые переходы при достижении границ кэшей L1d (~64 КБ), L2 (~1 МБ) и L3 (~32 МБ).
5. Использование дополнительного массива индексов в режиме random_precomputed удваивает фактически используемый объём памяти, что несколько смещает точки перехода между уровнями кэша по сравнению с режимом random_on_the_fly.
6. Полученные результаты подтверждают фундаментальный вывод: оптимальная производительность приложений, критичных к памяти, достигается при обеспечении локальности данных (data locality) — размещении активно используемых данных в кэшах верхних уровней иерархии.